Arquitectura y configuración del sistema RiceVi

Documento de diseño · hardware, red y flujo de datos Versión 1.3 — 23 de octubre de 2026, revisada el 24 (§5.1, `session_id`)

Este documento describe el sistema completo tal como va montado en la cosechadora: qué hardware lo compone, cómo se alimenta, cómo se comunica, cómo se identifican y guardan los datos y cómo se procesan. Corresponde al proyecto `config-ricevi-campo/`, que es el código que se ejecuta.

Los valores marcados como **estimados** son cálculos de dimensionamiento que todavía no se han medido sobre el montaje real. Están señalados uno a uno, y cada uno indica cómo comprobarlo.

> **Cambios de la versión 1.1.** La central ya no se alimenta de la máquina: lleva su propio paquete de baterías, y con él desaparecen la señal de contacto y el apagado por llave (§3.5). La iluminación de la tolva pasa a ser pulsada y comparte paquete con `CAM04` (§3.6). Se declara la limitación de la velocidad estimada por rueda (§5.3), se añade la validación pendiente de los ultrasónicos (§10) y se corrigen cifras de las §2.3 y §9.
>
> **Cambios de la versión 1.2 (hardware que llegó en octubre).** Los nodos de sensor pasan a **ESP32-C6 DevKitC**; el ultrasónico es el **RCWL-1655** en modo Trig/Echo, no el A02YYUW; el Hall es el módulo **HW-477** (A3144). Los nodos se alimentan con **4 pilas AA Ni-MH** en serie, directo al pin de 5 V, y **no llevan microSD**. La central se alimenta con una **batería LiFePO4 de 12 V y 12 Ah**, y el punto de acceso es un **router Tenda N301** a 9 V. El LED de la tolva es un módulo de inyección de 12 V con su propia batería de 9 V (§2, §3, §4.2, §5, §9, §10, §11).
>
> **Cambios de la versión 1.3 (velocidad por GPS).** La velocidad de avance ya no sale de un Hall en la rueda: la mide un **receptor GPS USB VK-162** (u-blox 7) conectado a la Pi, con la antena en el **techo de la cabina**. Sale el nodo `NH02` y quedan **siete** ESP32. La velocidad pasa de estimada a medida sobre el suelo, sin patinaje ni circunferencia que medir, y el *reel index* se calcula con ella. La posición que entrega el receptor **no se guarda** (§2, §4.1, §5.3, §5.4, §6, §9, §10, §11).

---

1 · Qué hace el sistema y en qué dos fases

RiceVi observa la operación de una cosechadora de arroz —el corte y lo que llega a la tolva— con cuatro cámaras, tres sensores y un receptor GPS, guarda esa observación de forma trazable, y —cuando existan modelos entrenados— la analiza a bordo para dar retroalimentación al operario.

El alcance es **percepción, diagnóstico y sugerencia**. El operario decide si modifica la máquina; el sistema no acciona nada.

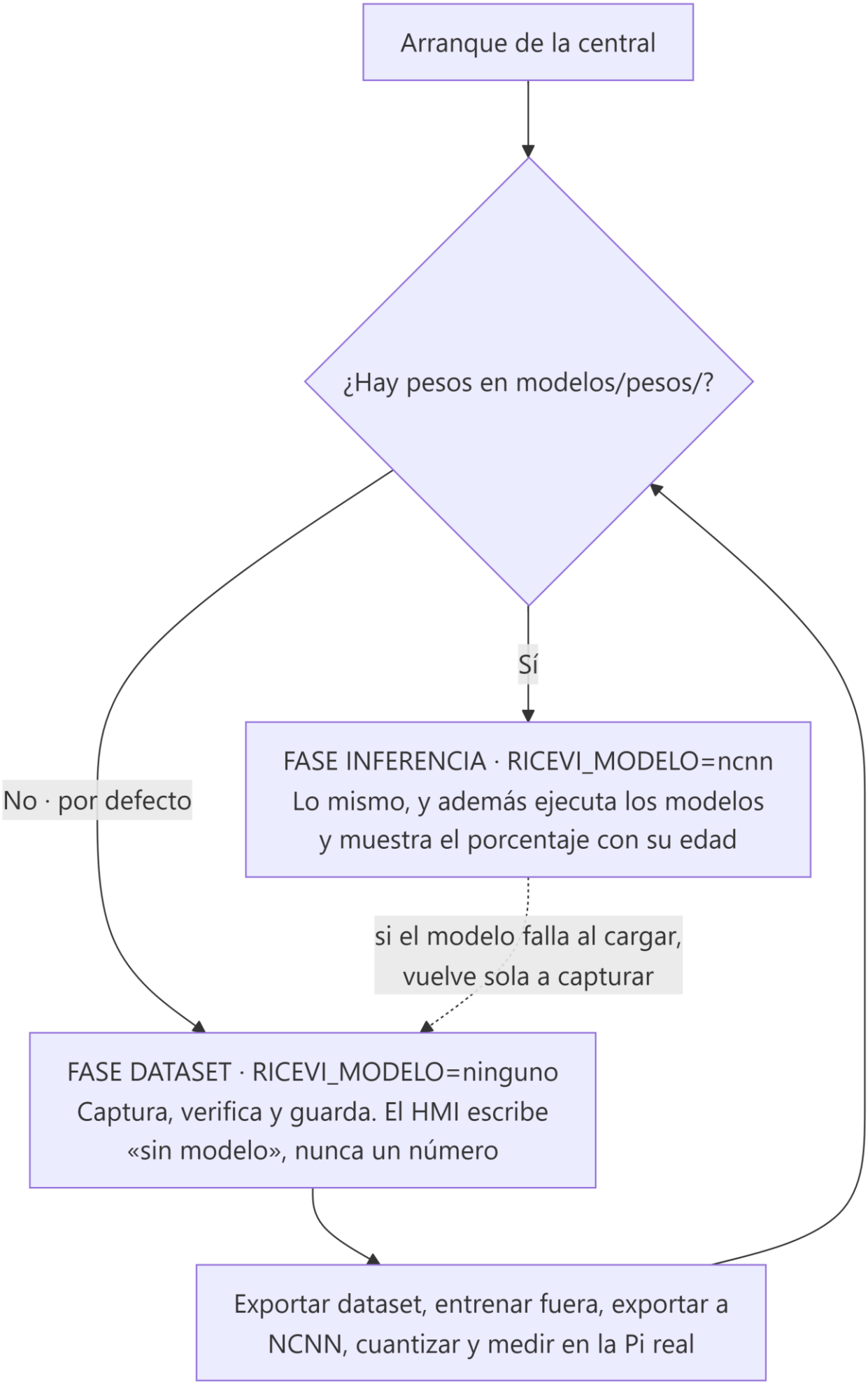
Como el sistema observa la operación tal como ocurre, sin provocar condiciones a propósito, las sugerencias tienen tres límites que conviene fijar desde ahora: salen de **reglas documentadas** —el manual de la máquina o la literatura—, no de patrones hallados en los propios datos; son **cualitativas** («considere subir la velocidad del molinete»), no valores objetivo; y **no se afirma** que seguirlas reduzca las pérdidas, porque eso exigiría un experimento controlado.

Subsistema	Qué observa	Qué produce
PRE-CORTE	Cultivo antes de entrar al cabezal	Clase de acame: erecto, moderado, severo
POST-CORTE	Franja recién cortada	Fracción visual de material útil residual
TOLVA	Superficie del material acumulado	Fracción visual de material no grano
Operación	Molinete, avance y distancia al suelo	RPM, velocidad sobre el suelo (GPS),

		alturas, *reel index*

La trilla queda fuera del alcance actual. Si más adelante sobra tiempo, podría añadirse etiquetando grano partido sobre las mismas fotos de la tolva, sin volver a campo: por eso conviene capturarlas con calidad suficiente desde el principio.

1.1 Las dos fases del mismo sistema


Arranque de la central
¿Hay pesos en modelos/pesos/?
Sí
No · por defecto
FASE INFERENCIA · RICEVI_MODELO=ncnn
Lo mismo, y además ejecuta los modelos y muestra el porcentaje con su edad
si el modelo falla al cargar, vuelve sola a capturar
FASE DATASET · RICEVI_MODELO=ninguno
Captura, verifica y guarda. El HMI escribe «sin modelo», nunca un número
Exportar dataset, entrenar fuera, exportar a NCNN, cuantizar y medir en la Pi real

Las dos fases

Fase	Qué hace	Cómo se activa
1 · Dataset	Captura, verifica y almacena. **No infiere**	`RICEVI_MODELO=ninguno` *(por defecto)*
2 · Inferencia	Lo mismo, y además ejecuta los modelos	`RICEVI_MODELO=ncnn` con los pesos instalados

La fase 1 es la primera y dura toda la campaña de captura. No hacen falta modelos para salir a campo: hacen falta datos para poder entrenarlos.

> **Regla de diseño que gobierna todo el sistema:** cuando no hay modelo, la pantalla **no muestra un número**. Muestra «sin modelo».

No es un detalle de interfaz. Un sustituto que devuelve un porcentaje plausible —por ejemplo, el ratio de píxeles verdes de la imagen— **no falla**: produce un valor con la forma correcta, en el rango correcto, junto a la etiqueta «material útil residual», y acaba en una tabla de la tesis sin que nadie lo detecte. Por eso el motor por defecto devuelve nulo, y el nulo se distingue del cero en todo el recorrido: en el contrato, en la base de datos y en la pantalla.

---

2 · Inventario de hardware

2.1 Los nueve equipos

ID	Equipo	Ubicación	Función
`RPI01`	Raspberry Pi 5, 8 GB + SSD NVMe 128 GB	Cabina	Recibe, verifica, almacena, infiere y sirve el HMI
`CAM01`	GOOUUU ESP32-S3-CAM (N16R8) con OV3660	Frente al cabezal	Cultivo antes del corte
`CAM02`	GOOUUU ESP32-S3-CAM (N16R8) con OV3660	Detrás del cabezal, izquierda	Resultado del corte
`CAM03`	GOOUUU ESP32-S3-CAM (N16R8) con OV3660	Detrás del cabezal, derecha	Resultado del corte
`CAM04`	GOOUUU ESP32-S3-CAM (N16R8) con OV3660 + LED pulsado	Interior de la tolva	Superficie del material acumulado
`NH01`	ESP32-C6 DevKitC + Hall HW-477 (A3144)	Eje del molinete	Pulsos → RPM
`NU01`	ESP32-C6 DevKitC + RCWL-1655	Extremo izquierdo del cabezal	Distancia → altura de cuchilla
`NU02`	ESP32-C6 DevKitC + RCWL-1655	Extremo derecho del cabezal	Distancia → altura de cuchilla
`GPS01`	Receptor GPS USB VK-162 (u-blox 7)	Techo de la cabina, cable USB a `RPI01`	Velocidad de avance sobre el suelo

Más dos equipos de apoyo: `AP01`, el punto de acceso Wi-Fi de 2,4 GHz (un router Tenda N301), y `HMI01`, una tablet o teléfono con un navegador.

Siete microcontroladores, uno por punto de captura. Cada sensor y cada cámara tiene su propio ESP32 junto a él: ESP32-S3 en las cámaras, porque necesitan la PSRAM para el JPEG, y ESP32-C6 en los sensores. El firmware es el mismo para las dos familias; los pines cambian con `CONFIG_IDF_TARGET_ESP32C6`.

El GPS no lleva microcontrolador. Va por USB a la Pi, que lo lee directamente: comparte el reloj de la central, no ocupa la red y no tiene batería que vigilar (§5.3). Hasta la versión 1.2 había un octavo nodo, `NH02`, con un Hall en la rueda; su placa ESP32-C6 queda de repuesto.

Nodo	Señal	Pines de la ESP32-C6
Hall	Salida del A3144 por un diodo 1N5819, con pull-up de 10 kΩ a 3,3 V y 100 nF a GND; el módulo se alimenta a 5 V	GPIO10
Ultrasónico	RCWL-1655 alimentado a 3,3 V, sin R7 (modo GPIO)	Trig GPIO11 · Echo GPIO10
Batería	Divisor 150 kΩ / 100 kΩ (todavía sin montar)	GPIO1
LED RGB de estado	El de la placa	GPIO8

2.2 Por qué un microcontrolador por sensor

Ventaja	Explicación
Cables cortos	Cada sensor queda a centímetros de su microcontrolador. Desaparecen las tiradas largas de pulsos y de eco por el chasis
Aislamiento de fallas	Un nodo caído se lleva un solo sensor, no todas las variables operativas
Montaje independiente	Cada caja se ubica donde conviene mecánicamente
Firmware simple	Cada nodo hace una cosa. El conteo de pulsos no comparte CPU con la medida de eco del ultrasónico

El coste es que hay ocho estaciones Wi-Fi compartiendo un canal (los siete nodos y el HMI), siete relojes que sincronizar y siete juegos de pilas que vigilar. Las tres cosas están resueltas y se explican en las secciones 4, 6 y 3.

2.3 La central

Elemento	Elección	Por qué
Computador	Raspberry Pi 5, 8 GB	Cuatro núcleos Cortex-A76 a 2,4 GHz. La RAM no es el recurso escaso; la CPU sí
Almacenamiento	SSD NVMe por PCIe en un HAT M.2; hoy de 128 GB, y la Pi arranca desde él	A 1 imagen/s por cámara son ~17 GB por jornada: una microSD ni tiene capacidad ni aguanta esa escritura a diario
Sistema de archivos	ext4 con *journaling*	Recuperación fiable ante un corte, a diferencia de FAT
Reloj	RTC con batería	Da hora civil fiable sin Internet. Sin él, los datos quedan marcados `utc_quality: desconocida`
GPS	Receptor USB VK-162 (u-blox 7) con antena integrada y base magnética, en el techo de la cabina	Mide la velocidad de avance sobre el suelo. Se lee por USB, sin `gpsd` ni servicios añadidos (§5.3)
Refrigeración	Disipador con ventilador activo	Bajo carga sostenida el SoC se estrangula a partir de 80 °C

Acelerador de IA	**Ninguno**	Decisión del proyecto. La sección 7.3 explica el presupuesto de CPU y el escalonado térmico que lo hace viable

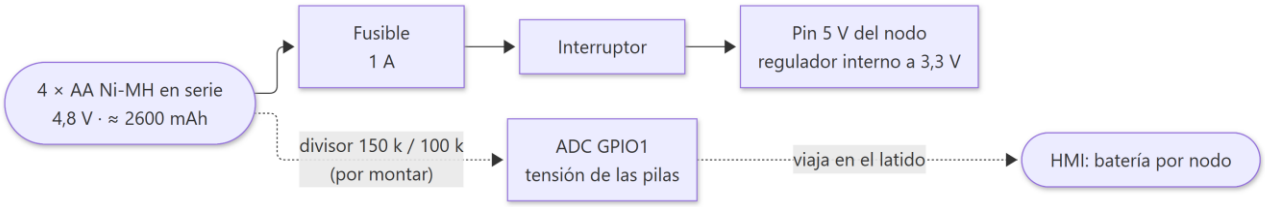
Por qué 512 GB. A 1 imagen/s por cámara, una jornada de ocho horas serían 115.200 fotografías y unos 17 GB; con `CAM04` a 0,5 img/s, como fija el plan, son unas 100.800 y unos 15 GB. **Con el SSD de 128 GB montado hoy** (≈ 119 GB útiles, compartidos con el sistema) caben unas seis o siete jornadas: hay que descargar y verificar el respaldo cada pocos días. Con 512 GB serían del orden de treinta.

---

3 · Alimentación

Nada se conecta a la instalación eléctrica de la cosechadora. La central lleva su propio paquete de baterías y cada nodo lleva el suyo. Así no hay que intervenir el cableado de una máquina ajena —que es, además, la parte del montaje que más permisos exige—, un fallo eléctrico de la cosechadora no afecta al sistema, y montar un nodo no obliga a tirar cable hasta él.

3.1 Los nodos: cuatro pilas AA Ni-MH en serie



Alimentación de un nodo

Elemento	Especificación
Pilas	4 × AA Ni-MH **en serie** (4S). La caja dice 3600 mAh; un AA Ni-MH real no pasa de unos 2800, así que se calcula con **2600 mAh** hasta medir un juego
Tensión	4,8 V nominales; 5,6 V recién cargadas; corte a 4,0 V (1,0 V por pila)
Energía útil estimada	≈ 10,6 Wh
Conversión	**Ninguna.** El portapilas entra directo al pin de 5 V; el regulador de la placa baja a 3,3 V
Protección	Fusible de 1 A e interruptor entre el portapilas y la placa. No hay placa de protección: el firmware avisa a 4,5 V y marca crítico a 4,35 V
Medida de tensión	Divisor 150 kΩ / 100 kΩ (factor 2,5) al GPIO1. **Aún no está montado**: hasta entonces el nodo publica la batería como nula
Carga	**Fuera de la máquina**, con el cargador de las pilas. Lo ideal son dos juegos: uno montado, otro cargando

Por qué AA Ni-MH. Es una decisión del equipo (octubre de 2026) que sustituye a las 18650: pilas comunes, cargador sencillo, sin elevador ni placa de protección por nodo, y una química que soporta mejor el calor y el abuso que el Li-ion. El precio es la energía: unos 10,6 Wh frente a los 18,9 del paquete de dos 18650, y por eso la autonomía de las cámaras baja a unas 11 horas (§3.2).

`CAM04` **ya no es una excepción.** El LED de la tolva tiene su propia batería de 9 V (§3.6), así que su nodo lleva el mismo portapilas que los demás.

3.2 Autonomía estimada

Las cuentas están a la vista para que puedan rehacerse con medidas propias, en lugar de creerse un número:

Rol	Consumo a 3,3 V	Ahorro Wi-Fi	Potencia en batería	Autonomía estimada
Cámara	200 mA	No	0,96 W	**≈ 11 h**
Cámara de tolva	200 mA	No	0,96 W	**≈ 11 h**; el LED gasta de su propia batería de 9 V (§3.6)
Nodo Hall	140 mA → 77 mA	**Sí**	0,37 W	**≈ 29 h**
Nodo ultrasónico	145 mA → 79 mA	**Sí**	0,38 W	**≈ 28 h**

Once horas es justo para una jornada larga. Si la cosecha pasa de diez horas, hay que cambiar las pilas de las cámaras a mitad de jornada.

> **Son estimaciones, no medidas.** Hay que confirmarlas con un multímetro en serie sobre un nodo real antes de dar la autonomía por buena. El script `config/censo.py` imprime el desglose completo del cálculo.

3.3 El ahorro de energía del radio: una decisión que no es uniforme

Desactivar el ahorro de energía del Wi-Fi mantiene el radio en recepción continua. Eso reduce la latencia —el nodo responde de inmediato en vez de esperar a la siguiente baliza— y **aproximadamente duplica el consumo**.

La decisión correcta es distinta para cada tipo de nodo, y sale de los números:

Nodo	Ahorro	Razón
Cámaras	**Desactivado**	El enlace es el cuello de botella de la latencia. Dormir el radio añade decenas de milisegundos impredecibles justo donde duele
Sensores	**Activado**	Su telemetría viaja por lotes de ~1 s y **cada muestra lleva su propio instante de captura**: el retraso de entrega no degrada la medida. Sale casi gratis y casi duplica la autonomía

Esa asimetría es un buen ejemplo de cómo el contrato de datos paga dividendos: como el instante de la medida viaja *dentro* de la muestra y no se deduce de cuándo llegó, retrasar la entrega no cuesta precisión.

El ajuste **no está compilado en el firmware**: llega en el plan que la central envía a cada nodo, de modo que cambiarlo es editar un archivo y reiniciar la central.

3.4 Seguridad y temperatura de las celdas

Esto merece una advertencia explícita, porque combina mal con el resto del montaje:

> **Ninguna batería debe ir en una caja sellada y oscura expuesta al sol.** Una caja así en un arrozal colombiano pasa con facilidad de 60 °C internos. Las Ni-MH de los nodos pierden capacidad y vida rápidamente a esa temperatura, y la batería de 9 V del LED y la LiFePO4 de cabina tampoco deben cargarse en caliente.

Medidas concretas:

Medida	Detalle
Ubicación	A la sombra siempre que sea posible; nunca en el punto más soleado
Caja	Color claro, y ventilada donde el ambiente lo permita
Carga	**Nunca dentro de la máquina**, y nunca en caliente
Pilas	Sin placa de protección: hay que respetar el aviso de batería del HMI y no descargarlas por debajo de 1,0 V por pila
Montaje	Portapilas de 4 AA con fusible e interruptor, **nunca pilas soldadas**
Fusible	1 A en serie con cada portapilas
Inspección	Revisar las celdas al cambiarlas: hinchazón, corrosión o calor anómalo
Sonda del ultrasónico	El RCWL-1655 no tiene hoja de datos oficial: su rango de temperatura está sin confirmar. A la sombra, igual que las pilas

3.5 La central

Elemento	Especificación
Fuente	**Batería LiFePO4 de 12,8 V y 12 Ah** (≈ 150 Wh) con BMS, compartida con `AP01`
Conversión	Convertidor 12 V → 5 V con salida USB-C de **5 A** para la Pi; un segundo convertidor 12 V → 9 V para el router Tenda N301
Consumo típico	≈ 9–12 W al 70 % de CPU con NVMe y Ethernet *(estimado)*; menos en fase de dataset, que no infiere. El GPS añade unos 0,25 W desde un USB de la Pi *(estimado)*, holgado dentro de los 600 mA que permite la Pi sin PD
Autonomía estimada	≈ 12 h con la Pi y el AP en fase de dataset; menos en fase de inferencia
Reserva local	1000 µF electrolíticos más cerámicos junto a la Pi
Protección	Fusible general junto a la batería, y uno por rama (Pi y router)
Apagado	**Manual**, antes de desconectar la batería

Por qué una LiFePO4 de 12 V y no el paquete 3S5P de la versión 1.1. Es una sola pieza de unos 150 Wh, en lugar de quince celdas que cargar una a una; tolera mejor el calor y admite miles de ciclos. Desde 12 V, la Pi y el router toman en conjunto del orden de 1 A. Su BMS corta en seco cuando la batería se agota, y eso equivale a desenchufar la Pi: hay que apagar antes.

Por qué 5 A, y por qué no una powerbank. Raspberry Pi especifica 5 V / 5 A para la Pi 5 mediante un perfil de carga propio, no porque consuma 25 W de forma continua, sino porque el sistema de gestión de energía necesita margen para los picos y **reacciona mal a la caída de tensión**. La mayoría de powerbanks entregan como mucho 5 V / 3 A: es la misma situación que un convertidor genérico de 3 A con cable fino, que produce reinicios por subtensión, el fallo más común y más difícil de diagnosticar en despliegues de este tipo.

El convertidor no habla USB-PD. Sin negociación PD, la Pi 5 avisa al arrancar y limita los USB a 600 mA. Con `usb_max_current_enable=1` en `config.txt` acepta la fuente de 5 A. Hay que medir que el convertidor no pase de 5,1 V y vigilar los avisos de subtensión.

Cómo se carga. Fuera de la máquina, con un cargador LiFePO4 de 14,6 V; con 2 A, una noche.

El apagado sigue siendo el fallo nuevo que introduce usar una Pi. Un ESP32 se desconecta en caliente casi impunemente; una Raspberry Pi no. Hay que parar el servicio y esperar a que la Pi se apague **antes** de desconectar o cambiar la batería. **Los nodos no llevan microSD:** mientras la central está apagada, los frames que capturan se pierden (el LED del nodo lo indica en ámbar). Por eso el cambio de batería se hace con la máquina parada, entre tolvadas, nunca en medio de una pasada.

3.6 La iluminación de la tolva

La tolva va cubierta, así que dentro está oscuro: el LED no compite con el sol y solo tiene que alumbrar un recinto cerrado. Eso abarata la iluminación.

Elemento	Especificación
Fuente de luz	Módulo de inyección de 12 V con 3 LED (cable rojo +, blanco −). Potencia probable de 1 a 1,5 W, **sin confirmar**
Alimentación	**Batería recargable de 9 V (PP3, 250 mAh) propia**, sin driver ni elevador. No toca el portapilas del nodo
Encendido	**Pulsado**, solo alrededor de cada foto: MOSFET IRLZ44N en lado bajo, puerta en G14 por 200 Ω y 100 kΩ a GND, fusible de 1 A
Firmware	**Pendiente:** `camara.ino` todavía no maneja el LED
Ventana	150–300 ms por foto, que llegará en el plan como cualquier otro parámetro
Montaje	Separado de la cámara, alumbrando en oblicuo

Por qué pulsado. `CAM04` toma una foto cada dos segundos. Una PP3 de 250 mAh con el LED siempre encendido duraría poco más de una hora; encendido solo durante la foto, el consumo baja unas diez veces.

El límite de los 9 V. Un módulo de 12 V alimentado a 9 V da bastante menos luz, y el brillo baja a medida que se descarga la batería. La autonomía real y la exposición hay que medirlas en el banco. Si no alcanza, el mismo cableado admite una batería de 12 V.

Por qué en oblicuo y separado de la cámara. Dentro de una tolva cerrada hay mucho polvo. Un LED junto al objetivo devuelve luz dispersa —el efecto de los faros en la niebla— y refleja brillos en el grano. La luz lateral, además, realza la textura, que es lo que distingue el grano de la paja y la cascarilla.

Cuidado con la sincronía. El driver de cámara guarda cuadros en memoria: si se enciende el LED y se captura de inmediato, la foto puede ser de antes del encendido. El orden es encender, descartar uno o dos cuadros, capturar y apagar. Hay que comprobar en el banco que **todas** las fotos guardadas salen iluminadas.

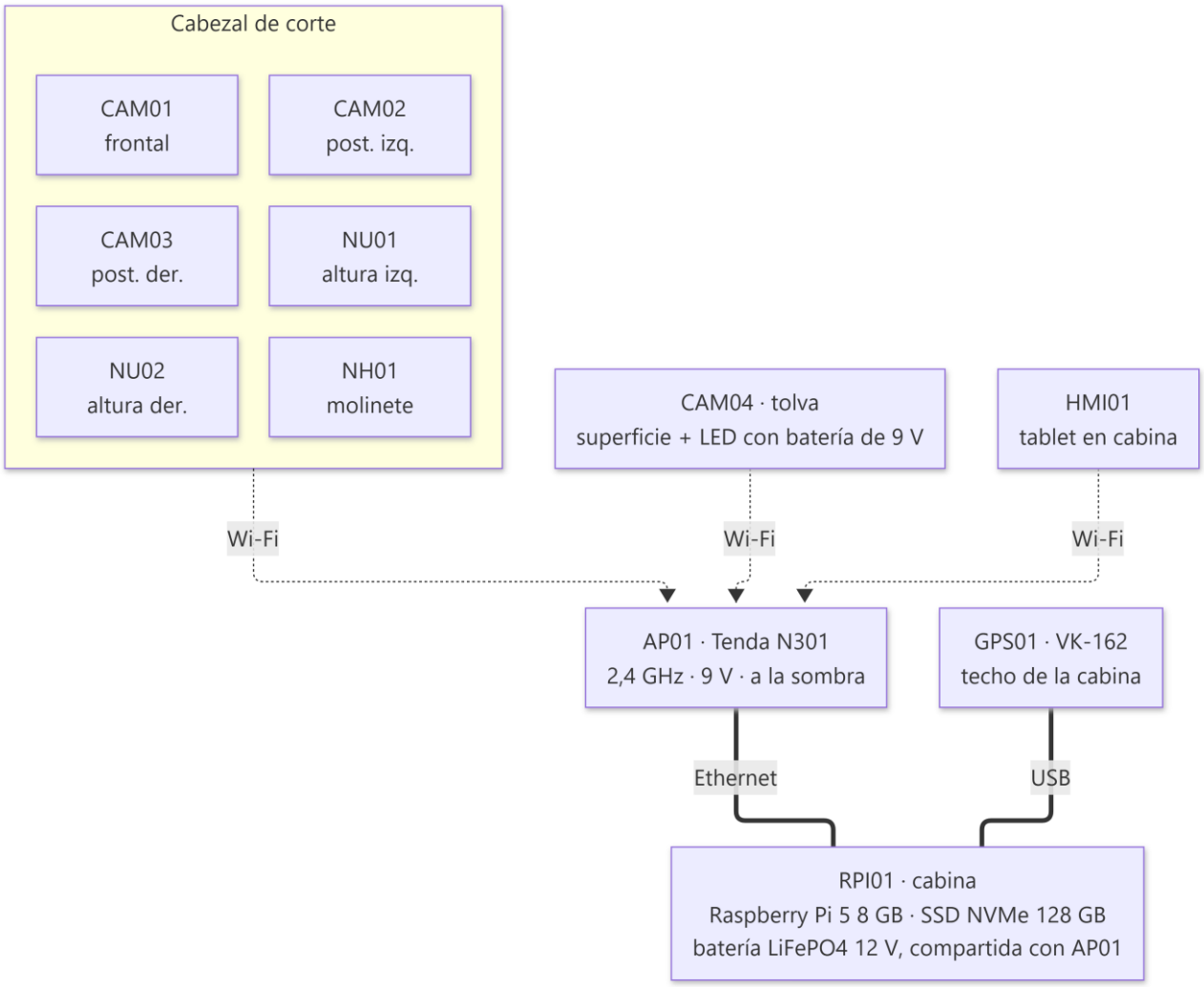
Lo que el LED no arregla. La superficie sube al llenarse la tolva y se acerca al LED: entre metro y medio y medio metro, la iluminación se multiplica por nueve. Con exposición manual fija, las fotos del final de la tolvada saldrán más claras que las del principio. Es la misma razón por la que el análisis debe acotarse según el nivel de llenado, ahora por dos motivos: resolución y brillo. La referencia de color sirve además para notar cuándo el polvo acumulado sobre el LED está quitando luz.

La batería va fuera de la tolva. Dentro solo la cámara y el LED. Dentro hay polvo de grano, que es combustible, y grano que puede golpear o enterrar la caja. Y conviene mirar dónde queda el motor en la máquina: en muchas cosechadoras está detrás o debajo de la tolva.

---

4 · Montaje físico y red

4.1 Dónde va cada cosa



Mapa físico

La decisión de montaje resuelve una tensión: **la Pi quiere estar fresca y la antena quiere estar cerca de los nodos.** Se separan.

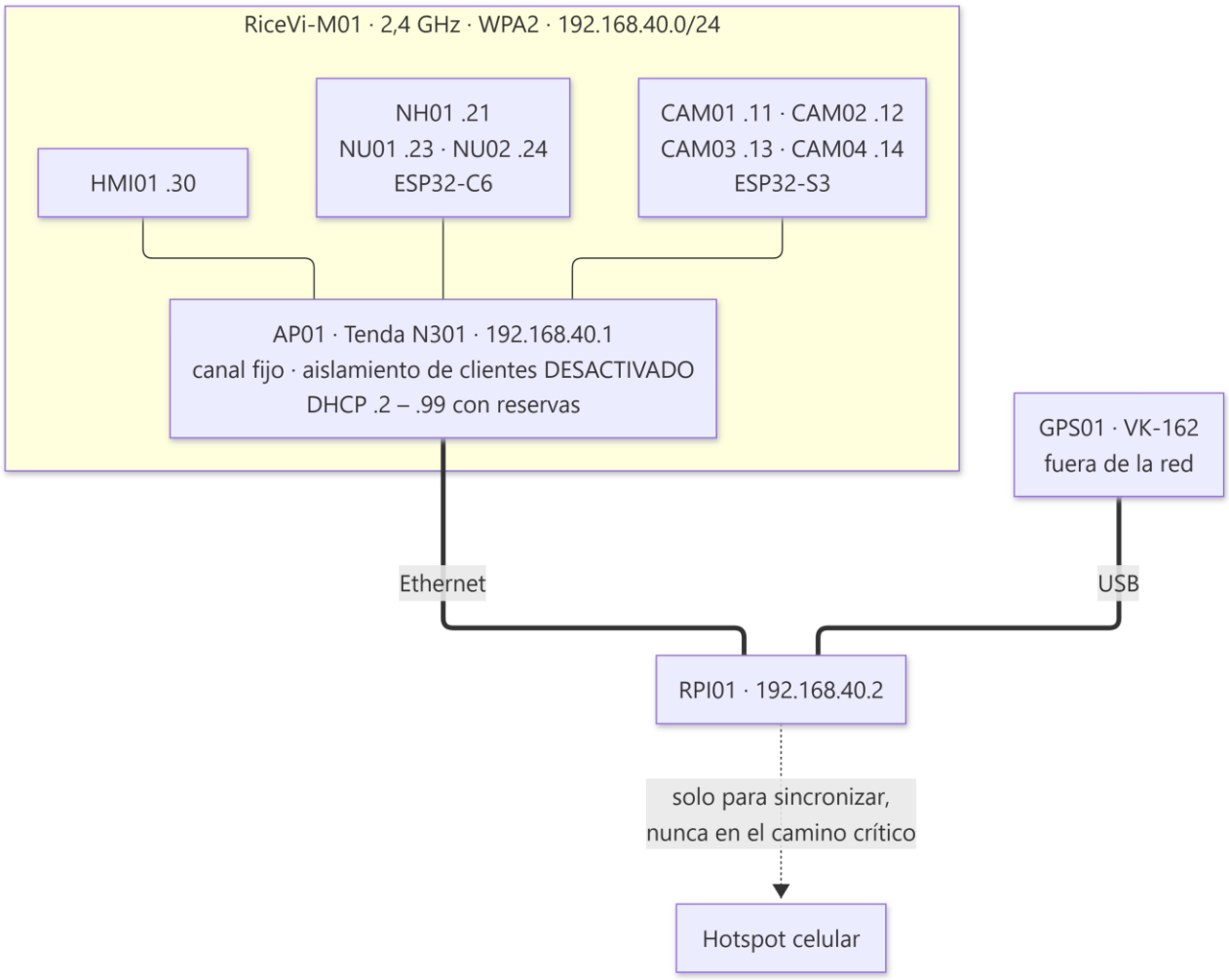
> **`RPI01` en cabina, `AP01` fuera y cerca del cabezal, unidos por Ethernet apantallado.**

El punto de acceso es pequeño, barato de reemplazar y tolera la intemperie mucho mejor que una Pi con SSD. La Pi vive en el sitio benigno, y el enlace entre ambos es cable, no radio.

Elemento	Requisitos de montaje
Caja de `RPI01`	Rejillas con filtro de polvo, ventilador, montaje antivibración, acceso a conectores
Caja de `AP01`	Estanca, antena orientada hacia los nodos, lo más alta posible, lejos de masas metálicas. Es un router Tenda N301 a 9 V; su alimentación baja de la batería de cabina, junto al cable Ethernet
Batería de la central	LiFePO4 en cabina, a la sombra, sujeta y accesible para cambiarla sin herramientas
GPS	Con su imán **en el techo de la cabina**, con cielo despejado y

	lejos de la antena del AP. El cable baja a un USB de la Pi por el mismo paso que el Ethernet, sujeto contra la vibración. Comprobar que el cable alcanza; si no, un alargador USB corto
Cajas de cámara	Visor transparente limpiable, **sujeción mecánica del conector FPC**, orientación fijada y documentada
LED de la tolva y su batería	El LED dentro, separado de la cámara y en oblicuo; **la batería de 9 V, fuera de la tolva** (§3.6)
Cajas de sensor	Estancas, con prensaestopas y alivio de tensión
Todas	Color claro, a la sombra cuando se pueda (ver 3.4), **etiquetadas antes de montar**

4.2 La red



Topología de red

Parámetro	Valor	Por qué
SSID	`RiceVi-M01`	Uno por máquina
Seguridad	WPA2-PSK	WPA3 puro puede no negociar con el ESP32
Banda	**Solo 2,4 GHz**	Ni el ESP32-S3 ni el ESP32-C6 **tienen radio de 5 GHz**. En «automático» el nodo no ve la red, y el síntoma es idéntico al de una placa averiada

Canal	**Fijo: 1, 6 u 11**	Un canal automático puede cambiar en marcha y desconectar a los siete a la vez
Ancho	20 MHz	40 MHz en 2,4 GHz añade interferencia sin beneficio
Aislamiento de clientes	**DESACTIVADO**	Si está activo, los nodos ven Internet pero **no ven a la Pi**. Es el fallo número uno y no da ningún mensaje de error
DHCP	Con reservas por MAC	Direcciones fijas sin grabar nada en el nodo
Portal cautivo	Ninguno	Un ESP32 no puede aceptar términos en una página

4.3 Router o celular: el número que decide

Elemento	Estaciones Wi-Fi
Cuatro cámaras	4
Tres nodos de sensor	3
`HMI01`	1
`RPI01`, solo si no va por cable	1
Total	**8 o 9**

El GPS no cuenta: va por USB a la Pi.

Punto de acceso	Clientes que admite	¿Sirve?
Router o AP dedicado	Decenas	**Sí, con holgura**
Hotspot de Android	Hasta unos 10, según fabricante	Justo, sin margen
Hotspot de iPhone	Unos 5, según modelo y operador	**No. No caben siete nodos**

Conclusión: router dedicado para campo. El hotspot de un teléfono sirve para pruebas parciales con dos o tres nodos en el taller, y como plan de contingencia, pero no para el sistema completo.

Si el router falla en campo, la Pi puede hacer de punto de acceso con su propia radio. Los nodos la encuentran por la tercera vía de descubrimiento. El coste es que la Pi pierde su salida a Internet mientras tanto, lo cual para capturar dataset es irrelevante.

4.4 Cómo encuentra un nodo a la central

Tres vías, probadas en orden, con `GET /salud` como prueba de vida:

Vía	Cuándo funciona
1 · IP fija `192.168.40.2`	Red de campo con reservas DHCP
2 · mDNS `ricevi-rasp-01`	Hotspot de móvil, o si la reserva falló
3 · Puerta de enlace	Solo si la Pi es además el punto de acceso

Se prueba `/salud` y no un socket a secas porque **que el puerto acepte una conexión TCP no significa que la central esté lista**: el servidor escucha antes de que el motor esté cargado.

4.5 Los cinco fallos de red, en orden de descarte

La primera comprobación que falla dice dónde está el problema.

#	Comprueba	Si falla
1	El nodo se asocia al SSID	Banda equivocada, contraseña, o fuera de cobertura
2	Recibe IP por DHCP	Servidor agotado o reserva mal escrita
3	Alcanza `192.168.40.2`	**Aislamiento de clientes activado**, o la Pi caída
4	Recibe `200` de `/salud`	Cortafuegos, o servicio no arrancado
5	Sigue conectado **en movimiento**	Cobertura. Es el que no se detecta en el taller

El quinto merece atención: un nodo en el extremo del cabezal, con el chasis metálico entre su antena y la del AP, puede asociarse perfectamente parado y perder el enlace cada vez que la máquina gira. La prueba es circular con el sistema en marcha y mirar la tasa de reintentos por nodo, no la señal en reposo.

---

5 · Cómo se identifican los datos

Esta sección define el etiquetado. Si algo de aquí cambia después de empezar a capturar, el dataset queda partido en dos mitades que no se pueden comparar.

5.1 Los identificadores

Identificador	Forma	Quién lo asigna	Cuándo cambia
`machine_id`	`M01`	Fijo del proyecto	Si hay una segunda cosechadora
`session_id`	`R001`, `R002`…	La central al arrancar	En cada arranque de la central, normalmente uno por jornada. Nunca se repite, aunque la corrida anterior no guardara frames
`node_id`	`CAM01`…`NU02`	El censo, grabado en NVS	**Nunca.** Es la identidad del equipo
`sensor_id`	`HALL01`, `US01`, `GPS01`…	El censo	**Nunca.** Sigue al sensor aunque cambie de placa
`boot_id`	`B01`, `B02`…	El nodo	En cada arranque
`seq`	Entero continuo	El nodo	Se reinicia solo con `boot_id` nuevo
`frame_id`	`M01-R001-CAM02-B03-000036`	El nodo	Uno por imagen
`config_version`	`cfg-campo-1.1.0`	La central	Al cambiar un parámetro que afecta a los datos
`censo_version`	`censo-1.1.0`	El censo	Al cambiar una posición, una IP o una calibración

5.2 Tres reglas del `frame_id`

```
^[A-Z0-9]{3}-R\d{3}-(CAM|NH|NU)\d{2}-B\d{2,4}-\d{6}$
```

1. **`seq` va con seis dígitos y ceros a la izquierda.** Así el orden alfabético de los nombres de archivo coincide con el orden temporal.
2. **`seq` no se reinicia dentro del mismo `boot_id`.** Es lo que permite detectar huecos: si llegan el 35 y el 37, se perdió el 36 y queda contado. Sin secuencia continua, una pérdida es invisible.
3. **Lo genera el nodo, no la central.** Es lo que hace posible el reintento idempotente: si la confirmación se pierde y el nodo reenvía, la central reconoce el mismo identificador y no crea un duplicado.

Por qué `boot_id`. Sin él, el frame 36 de hoy y el 36 de después de un reinicio serían el mismo identificador, y uno pisaría al otro.

`session_id` no sirve para agrupar el dataset. Si la central se reinicia a mitad de jornada —al cambiar el paquete de baterías, por ejemplo— abre una sesión nueva, y esa jornada quedaría partida en dos. Por eso la división en entrenamiento, validación y prueba se agrupa por **fecha de la jornada** y no por sesión (manual de campo, §5).

Por qué `sensor_id` no es el `node_id`. Si la placa `NU01` se avería y se sustituye, el sensor montado sigue siendo `US01`. La calibración, el `offset_b_mm` y el histórico pertenecen al sensor, no a la placa que lo lee.

5.3 Qué magnitud produce cada sensor

Cada nodo publica **lo que mide y lo que deriva, por separado**:

Sensor	Magnitud	Unidad	¿Derivada?	Fórmula
`HALL01`	`pulsos_ventana`	count	No	Conteo directo del contador por hardware
`HALL01`	`reel_rpm`	rpm	**Sí**	`60·N / (P·Δt)`
`US01` `US02`	`distance`	mm	No	Duración del eco del RCWL-1655 × velocidad del sonido / 2
`US01` `US02`	`header_height`	mm	**Sí**	`distance − offset_b_mm`
`GPS01`	`ground_speed`	m/s	No	Velocidad sobre el suelo de la trama RMC: nudos × 0,5144
`GPS01`	`ground_speed_filtrada`	m/s	**Sí**	Media de los últimos 3 s; 0 por debajo de 0,2 m/s (`reposo`)
`GPS01`	`satelites`, `hdop`	count, —	No	Trama GGA: calidad de la solución

Por qué se guarda también la cruda. Si dentro de tres meses se corrige el número de imanes del molinete, con los pulsos guardados las RPM de toda la campaña se recalculan. Sin ellos, hay que repetir la campaña. Los pulsos son el dato; las RPM son una interpretación del dato. Con el GPS pasa lo mismo: si se cambia el filtro, se recalcula desde `ground_speed`.

La velocidad la mide el GPS. El receptor la calcula con el desplazamiento Doppler de las señales de los satélites, que es mucho más estable que derivar posiciones sucesivas, y lo que mide es lo que la máquina avanza de verdad sobre el suelo. Desaparecen los dos problemas del Hall de rueda que se usó hasta la versión 1.2: el montaje en una rueda metida en agua y lodo, y el patinaje, que convertía la velocidad en una estimación sin referencia con la que contrastarla. La hoja del u-blox 7 da del orden de 0,1 m/s de precisión en velocidad; las correcciones SBAS que anuncia el receptor (WAAS/EGNOS) no cubren Colombia de forma fiable, pero la velocidad Doppler no depende de ellas.

Lo que el GPS sí exige, y hay que vigilar:

Limitación	Consecuencia
Necesita ver el cielo	Bajo árboles, junto a un galpón o con la antena mal puesta pierde la solución. Entonces la velocidad es **nula** (`sin_fix`), nunca la última conocida
Arranque en frío de uno o dos minutos	Hay que encenderlo al aire libre antes de empezar a capturar; `verificar_campo.py` no da el visto bueno sin solución
A paso muy lento el Doppler tiembla	Por eso la filtrada promedia 3 s y publica 0 con la máquina detenida. El precio es un retraso de alrededor de un segundo y medio
Una solución por segundo	El molinete manda diez muestras por segundo; el emparejamiento lo resuelve (abajo)

La validación en campo es corta: recorrer un tramo medido con cinta a ritmo constante y comparar con la media del GPS, y ver cuánto marca con la máquina detenida (manual de campo, §7.2).

La posición no se guarda. La trama trae latitud y longitud; la central las usa solo para validar la trama y las descarta. En la base no queda dónde estaba el lote: el GPS del sistema solo mide la velocidad.

El *reel index* no lo calcula ningún nodo: `NH01` tiene las RPM y el GPS la velocidad. Lo calcula la central con la última velocidad del GPS y la muestra de RPM **más cercana a ese instante**, y guarda **cuánto se separan en el tiempo**, porque un índice calculado con muestras separadas dos segundos no es el mismo dato que uno con veinte milisegundos. Como el GPS va en la propia central, su instante ya está en el reloj de la sesión: la única incertidumbre es la de la sincronía de `NH01`, más el retraso de la trama del receptor, que se declara en `sync_uncertainty_ms` (100 ms).

5.4 Calidad: la regla del nulo

Cada muestra lleva dos campos que hacen cosas distintas:

- **`calidad`** decide si el dato se usa: `buena`, `dudosa`, `invalida`.
- **`codigo_calidad`** dice **por qué**, y es lo que permite diagnosticar.

Código	Calidad	Significado
`ok`	buena	Lectura normal dentro de rango
`fuera_rango`	inválida	Fuera del rango físico declarado
`checksum`	inválida	Solo para un sensor de trama serie (el A02YYUW de la versión 1.1); el RCWL-1655 no lo produce
`timeout`	inválida	No llegó eco dentro del plazo (`timeout_eco_us`)
`sin_pulsos`	dudosa	Ventana sin pulsos: parada real o sensor caído

`eco_dudoso`	dudosa	Distancia plausible pero inestable
`sin_calibrar`	inválida	Falta la constante para derivar la magnitud
`saturado`	dudosa	Valor en el límite superior del rango
`sin_fix`	inválida	El GPS no tiene solución: no hay velocidad
`hdop_alto`	dudosa	GPS con geometría pobre (HDOP por encima de 5)
`reposo`	buena	Velocidad por debajo de 0,2 m/s: máquina detenida, se publica 0

Por qué hacen falta los dos. Una tasa de inválidas del 20 % con `calidad` a secas no se puede diagnosticar. Con el código se sabe si es `checksum` —cableado, ruido eléctrico, conector flojo— o `fuera_rango` —geometría de montaje, vegetación, terreno—. Son dos averías distintas con dos soluciones distintas.

Y la regla que no se negocia:

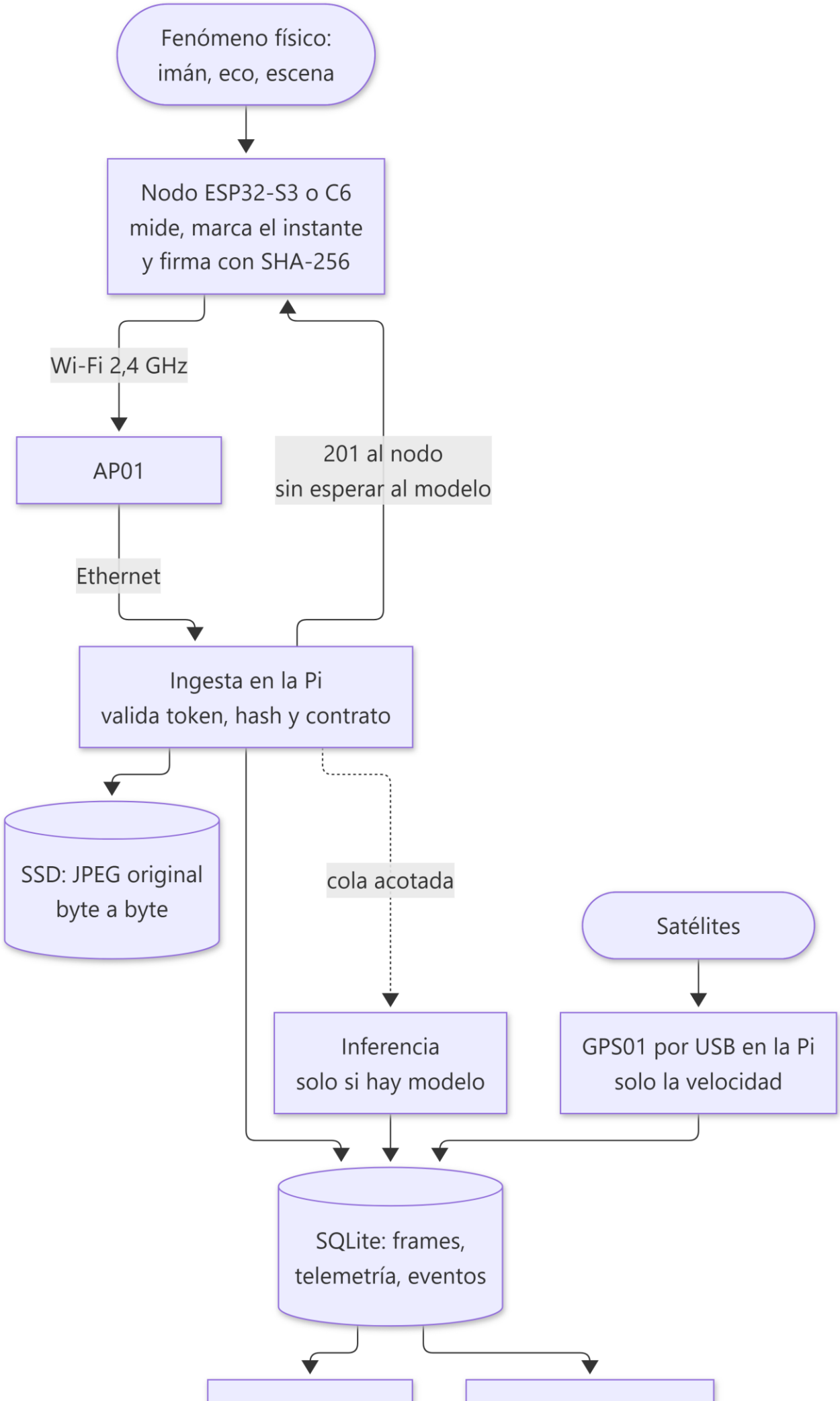
> **Cuando no hay medida, el valor es nulo.** Nunca cero, nunca el último valor conocido, nunca un valor por defecto.

Un ultrasónico sin eco que devuelve `0` mete en el dataset una altura de cuchilla de cero milímetros indistinguible de una medida real. Al entrenar, el modelo aprende de ese cero. Es el tipo de contaminación que no se detecta mirando gráficas y que invalida conclusiones meses después.

La regla se hace cumplir en el contrato: **una muestra con valor nulo y calidad `buena` se rechaza**, porque es un error de firmware y no un estado del mundo.

---

6 · Cómo viajan y se guardan los datos


Fenómeno físico:
imán, eco, escena
Nodo ESP32-S3 o C6
mide, marca el instante
y firma con SHA-256
Wi-Fi 2,4 GHz
AP01
201 al nodo
sin esperar al modelo
Ethernet
Ingesta en la Pi
valida token, hash y contrato
SSD: JPEG original
byte a byte
cola acotada
Satélites
Inferencia
solo si hay modelo
GPS01 por USB en la Pi
solo la velocidad
SQLite: frames,
telemetría, eventos

6.1 El protocolo

Todo va por HTTP sobre la red local, con un **token por nodo** en la cabecera. Ocho endpoints:

Endpoint	Emisor	Qué hace
`POST /v1/nodes/register`	Nodo	Se presenta y recibe sesión, plan y ranura de reloj
`POST /v1/time/exchange`	Nodo	Intercambio de cuatro tiempos
`GET /v1/nodes/{id}/plan`	Nodo	Ritmo, resolución, ROI y calibración
`POST /v1/upload-slots`	Cámara	Pide turno de subida
`POST /v1/frames`	Cámara	Imagen y metadatos, en una sola operación
`POST /v1/telemetry`	Sensor	Lote de muestras
`POST /v1/events`	Todos	Latido, batería, descarga de tolva, error
`GET /api/*`	HMI	Estado, resultados, trazabilidad

El nodo no decide casi nada. El ritmo de captura, la resolución, el ROI, la ranura de reloj, los umbrales de batería y el ahorro de radio llegan en el plan. Cambiar el ritmo de una cámara es editar un archivo y reiniciar la central, no desmontar una caja del cabezal. Siete veces.

6.2 El turno de subida

Siete nodos comparten un canal. Si las cuatro cámaras suben a la vez no van cuatro veces más rápido: compiten, colisionan, retransmiten, y las cuatro tardan más que si hubieran ido en fila. El control de admisión serializa las subidas.

Hay un segundo efecto, menos obvio: mientras las cámaras compiten, **la telemetría también compite**. Los nodos de sensor no piden turno —su tráfico es pequeño— pero si el aire está saturado por cuatro imágenes simultáneas, sus lotes se retrasan. El turno protege también el dato que da contexto a las imágenes.

Los turnos **caducan**: si un nodo pide turno y se cuelga o se queda sin batería, sin caducidad bloquearía a los demás para siempre.

6.3 La regla central del sistema

> La central responde `201` en cuanto el archivo está en disco y su fila en la base. **Encola la inferencia después, sin esperarla nunca.**

Si la cámara esperara al resultado del modelo, un pico de carga de CPU se propagaría hacia atrás hasta detener la captura. Un problema de diagnóstico se habría convertido en pérdida de dataset.

Cuando el sistema se satura se pierde diagnóstico, nunca dataset. Y el descarte queda contado, que es lo que convierte una degradación silenciosa en un número medible.

6.4 Integridad

Cada imagen viaja con su SHA-256, calculado por el nodo. **La central lo recalcula sobre los octetos recibidos**: el hash del nodo prueba lo que el nodo cree haber enviado; recalcularlo prueba que el viaje no los alteró.

El archivo se escribe a un nombre temporal, se fuerza a disco y se renombra. El renombrado dentro del mismo sistema de archivos es atómico, así que **nunca existe un `.jpg` a medio escribir**: si la Pi se apaga en mitad, queda un temporal que se barre, no un JPEG truncado que parece válido.

Lo que el hash **no** garantiza, y conviene decirlo: que la escena sea útil, que esté enfocada o que la lente no estuviera sucia. Es integridad de octetos, no calidad de imagen.

6.5 Los tres casos de un identificador repetido

Un nodo que no recibe la confirmación reenviará el mismo frame. Es lo correcto por su parte: no puede saber si llegó.

Situación	Respuesta	Por qué
Mismo ID, mismo hash	`200` con el estado existente	Reenvío legítimo. No duplica
Mismo ID, hash distinto	`409`	Dos contenidos reclaman la misma identidad: es un fallo de firmware y hay que verlo
ID nuevo	`201`	Camino normal

6.6 Qué se guarda y dónde

```
/datos/ricevi/
├── ricevi.db                    SQLite en modo WAL
├── frames/R001/CAM02/*.jpg      los octetos originales, sin recomprimir
├── derivados/                   máscaras y superposiciones, borrables
└── export/                      CSV para análisis
```

Tabla	Contenido
`sesiones`	El **acta**: instantánea completa de la configuración con la que se capturó
`nodos`	Los siete, con batería, RSSI y última vez vistos
`frames`	Una fila por imagen, con ruta, hash y tiempos
`telemetria`	Una fila por magnitud, con crudos y código de calidad. Las del GPS llevan `node_id` `RPI01` y `sensor_id` `GPS01`
`estado_maquina`	Estado operativo fusionado, con la edad de cada campo
`resultados`	Una fila por inferencia, con modelo, versión y tiempos por etapa
`eventos`	Arranques, cortes, batería baja, cambios de estado térmico
`intercambios_tiempo`	Todos los intercambios de reloj, para medir la deriva

Los `.jpg` son los octetos recibidos, byte a byte. Nunca se recomprimen. Cualquier versión anotada va a `derivados/`, que se puede borrar sin tocar el dataset.

`estado_maquina` es **derivada**: se puede borrar y regenerar entera desde `telemetria`. Eso significa que un error en esa lógica no destruye nada.

6.7 Sincronía de relojes

Cada nodo tiene un reloj que deriva, y la red añade retraso variable. Coincidir en la hora de recepción no basta.

El intercambio de cuatro tiempos estima el desfase:

$$\text{offset} \approx [(t_2 - t_1) + (t_3 - t_4)] / 2 \qquad \text{RTT} = (t_4 - t_1) - (t_3 - t_2)$$

El término $(t_3 - t_2)$ resta lo que la central tardó en procesar, de modo que el RTT mide solo el viaje. Importa: con la CPU cargada de inferencia, sin ese término se confundiría carga de CPU con mala red.

Se repite siete veces y **se conserva el de menor RTT**: es el que menos encolado ha sufrido, y por tanto aquel en el que el supuesto de simetría ida/vuelta es menos falso.

El supuesto hay que declararlo en la tesis: la fórmula asume que ida y vuelta tardan lo mismo. En Wi-Fi eso es falso en general. Por eso cada registro lleva `sync_uncertainty_ms` y el desfase no se presenta como exacto.

Los siete nodos sincronizan **escalonados**, cada uno en su ranura de un ciclo de 30 segundos, para no producir una ráfaga periódica que compita con las imágenes. La ranura es fija por nodo: si se asignara por orden de llegada, un nodo que se reinicia caería en la ranura de otro. El GPS no necesita sincronía: lo lee la propia central y lo sella con su reloj.

---

7 · Cómo se procesa

7.1 En fase de dataset

No se procesa: se guarda. La tubería se ejercita igual —el frame entra en la cola, se mide el tiempo de espera, se registra el resultado— y lo único que no hay es un número inventado.

Eso produce, además, la **línea de base de latencia** contra la que se comparará el coste real del modelo cuando llegue.

7.2 En fase de inferencia

Tarea	Tipo de problema	Cámara	Ritmo
Acame	Clasificación ordinal, 3 clases	`CAM01`	1 de cada 2 frames
Post-corte	Segmentación semántica	`CAM02` `CAM03`	1 de cada 1
Tolva	Segmentación semántica	`CAM04`	1 de cada 3

Captura e inferencia son dos ritmos distintos. El dataset se queda con todas las imágenes; solo el diagnóstico se ralentiza. Una imagen que hoy no se infiere no se pierde: está en el disco y puede procesarse en diferido.

Segmentación semántica y no de instancias. Para obtener un porcentaje de píxeles, lo natural es la semántica. Un modelo de instancias produce objetos individuales que luego habría que unir para no contar píxeles dos veces, y paga un coste por información que el indicador no usa.

NCNN y no PyTorch. Sobre Raspberry Pi 5 con YOLO11n a 640 × 640: NCNN 93,9 ms, ONNX 181,8 ms, TensorFlow Lite 324,2 ms, PyTorch 524,8 ms. Más de cinco veces de diferencia, sin coste. Las cifras proceden de la guía de Raspberry Pi de Ultralytics y hay que citarlas con su fecha de consulta, porque la tabla cambia con cada versión. Son además de **detección**: la clasificación de acame y las dos segmentaciones tendrán su propio coste, que habrá que medir en la Pi real (§10).

La trilla no está en esta tabla porque queda fuera del alcance actual. Si más adelante se entrena, saldría de las mismas fotos de `CAM04` con una clase de grano partido, sin cámara nueva.

7.3 El presupuesto de CPU, y el lazo térmico

Sin acelerador, la inferencia consume una fracción grande de la CPU. Eso genera calor, y el calor genera un lazo que no se estabiliza solo:

```
CPU al 70 % → sube la temperatura → el SoC se estrangula a 80 °C
→ cada inferencia tarda más → la cola crece → CPU al 100 % → más calor
```

La respuesta es **degradar deliberadamente antes de que lo haga el hardware**:

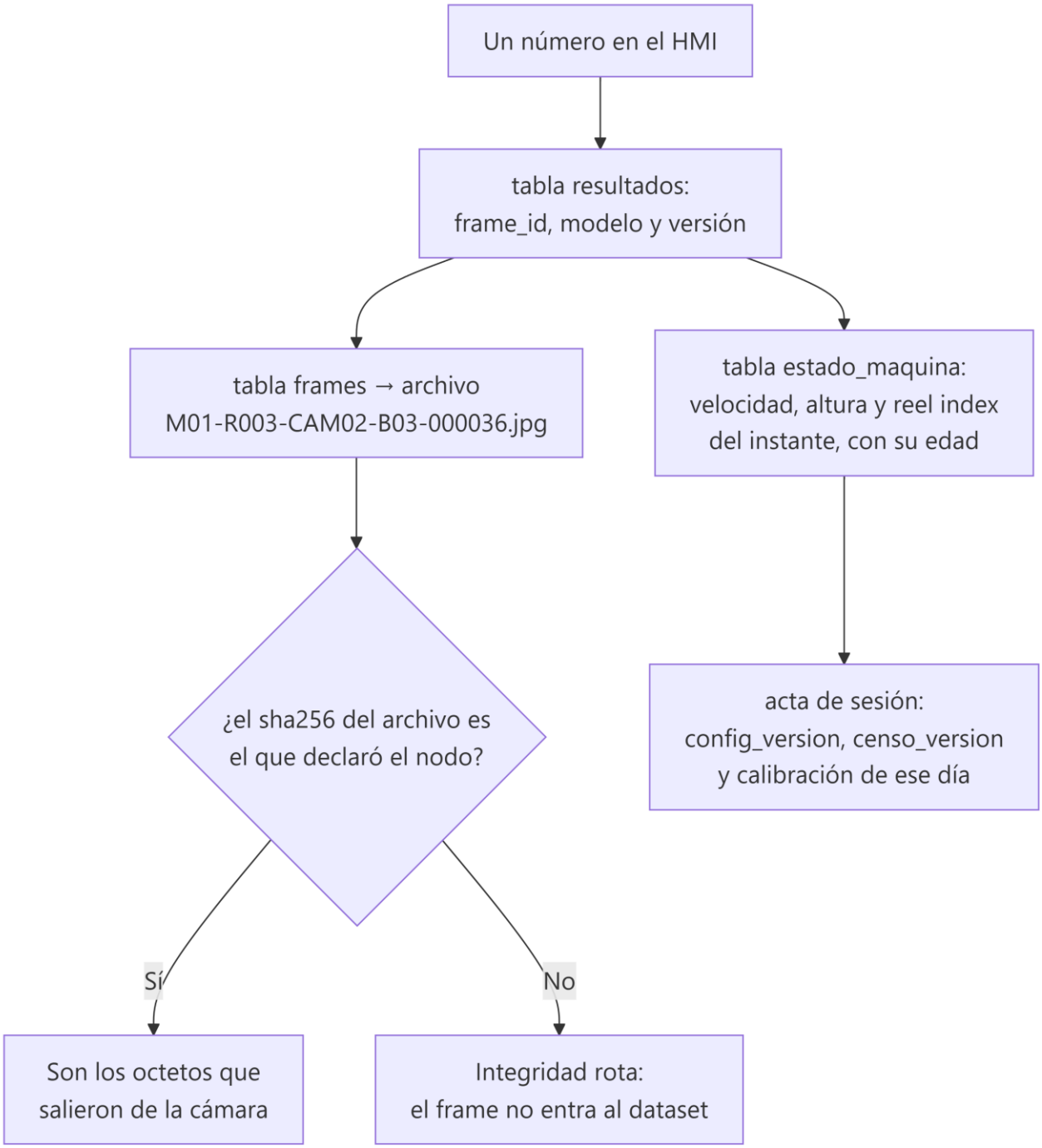
Temperatura del núcleo	Acción
< 65 °C	Ritmos normales
65 – 72 °C	La tolva baja a la mitad
72 – 78 °C	Todas las cámaras a la mitad
78 – 82 °C	Solo post-corte, a un cuarto
> 82 °C	**Inferencia en pausa**

> **La captura nunca se degrada por temperatura.** Solo el diagnóstico.

Cada cambio de escalón se registra como evento, para que un análisis posterior pueda explicar por qué una jornada tiene huecos de diagnóstico.

---

8 · Trazabilidad



Cadena de trazabilidad

La pregunta que el sistema tiene que poder responder meses después es:

> *«El HMI mostró 12,86 % a las 13:42. ¿De qué imagen salió, con qué modelo, en qué condiciones de máquina, y son esos los octetos originales?»*

Paso	Dónde se mira
1	`resultados` → `frame_id`, modelo, versión, tiempos por etapa
2	`frames` → ruta, `sha256`, instante de captura
3	El archivo en disco → los octetos

4	Recalcular el hash → confirma que nadie los tocó
5	`estado_maquina` → contexto operativo, con la edad de cada campo
6	`sesiones` → el acta: `config_version`, `censo_version` y calibración de ese día

Si el paso 4 falla, el frame no entra al dataset. No se arregla ni se acepta con una nota: se marca y se excluye. Un archivo cuyos octetos no coinciden con lo que declaró el nodo no se sabe qué es.

El acta de sesión importa más de lo que parece: si una jornada se capturó con una calibración provisional o con una MAC sin registrar, el acta lo dice. Sin ella, dos tandas con el mismo `config_version` pero distinto montaje parecerían comparables y no lo son.

---

9 · Lista de materiales

Precios de referencia en dólares de septiembre de 2026, **estimaciones para dimensionar**, sin aranceles ni IVA. Los ítems que cambiaron en octubre llevan el precio en pesos de la cotización del equipo cuando se conoce; los que no, se marcan *por confirmar*. **El total hay que recalcularlo** con la cotización.

9.1 La central

Cant.	Ítem	Ref.
1	Raspberry Pi 5, 8 GB	US$ 80
1	Disipador con ventilador activo	US$ 5
1	HAT M.2 para NVMe (GeekPi N04)	US$ 15
1	SSD NVMe 128 GB	*por confirmar*
1	Portapilas y pila CR2032 para el RTC	US$ 5
1	Receptor GPS USB VK-162 (u-blox 7) con base magnética	*por confirmar*
1	microSD de 64 GB con el sistema, como copia de rescate	US$ 8
1	Caja ventilada con filtro y antivibración	US$ 25

9.2 Nodos, sensores y baterías

Cant.	Ítem	Ref.
4	GOOUUU ESP32-S3-CAM V1.5 (N16R8) con OV3660	US$ 70 (dos paquetes de 3)
4	ESP32-C6 DevKitC (tres en uso; la de `NH02` queda de repuesto)	*por confirmar*
—	Envío a Colombia	US$ 20
2	Ultrasónico RCWL-1655	*por confirmar*
2	Módulo Hall HW-477 (A3144) + imanes (uno en el molinete, otro de repuesto)	US$ 22
8	Kit de 4 pilas AA Ni-MH + cargador (siete en uso, uno de recambio)	*por confirmar*

8	Portapilas de 4 AA + fusible de 1 A + interruptor (siete en uso)	*por confirmar*
—	Diodos 1N5819, resistencias de 10 kΩ, 150 kΩ y 100 kΩ, condensadores de 100 nF	*por confirmar*
8	Caja IP65 con prensaestopas (siete en uso)	US$ 56
1	Módulo LED de inyección de 12 V + referencia de color	US$ 20
1	MOSFET IRLZ44N, resistencias de 200 Ω y 100 kΩ, fusible de 1 A	US$ 10
1	Batería recargable de 9 V (PP3, 250 mAh) + cargador	*por confirmar*
—	Cable, conectores y herrajes	US$ 60

Un juego de pilas por nodo. Con siete nodos en uso, de las 32 pilas sobra un juego de recambio. Un segundo juego completo, para trabajar dos jornadas seguidas sin esperar cargas, serían otras 28.

9.3 Red y energía

Cant.	Ítem	Ref.
1	Router Tenda N301 (2,4 GHz, 9 V)	COP 41.000
1	Caja estanca para el AP	US$ 15
1	Cable Ethernet apantallado, con par de alimentación	US$ 12
1	Batería LiFePO4 12,8 V 12 Ah con BMS	COP 212.518
1	Cargador LiFePO4 14,6 V, 2 A	COP 52.271
1	Convertidor 12 V → 5 V USB-C 5 A, para la Pi	COP 29.388
1	Convertidor 12 V → 9 V 5 A con conector 5521, para el router	COP 39.195
1	Portafusibles con fusibles de 3 a 15 A	*por confirmar*
1	Capacitancia y protección	US$ 30

Sin servicio de nube recurrente: la inferencia es local.

Por qué una batería de 12 Ah. Con la Pi y el router en fase de dataset el sistema de cabina gasta del orden de 11 W en batería; unos 150 Wh dan unas doce horas. La de 7 Ah se descartó porque no llegaba a una jornada.

---

10 · Lo que falta medir antes de publicar

Cuatro constantes se miden a mano, no con código. **El sistema captura dataset válido sin ellas**, porque guarda siempre la magnitud cruda, pero hasta que estén medidas y anotadas, `reel_rpm`, `header_height` y `reel_index` **no se pueden publicar**. La velocidad del GPS no necesita ninguna.

Constante	Cómo se mide
`pulsos_por_vuelta` de `HALL01`	Contar los imanes del eje del molinete y comprobar con una

	vuelta a mano que el nodo cuenta exactamente ese número
`radio_molinete_m`	Del centro del eje a la **punta de la púa**, máquina parada y bloqueada
`offset_b_mm` de `US01`	De la cara del sensor al plano de la cuchilla
`offset_b_mm` de `US02`	Igual, lado derecho. **No asumir que es el mismo número**

Y una comprobación que no es una constante sino una duda de fondo:

Comprobación	Cómo se hace
Que los ultrasónicos midan hasta el suelo y no hasta el rastrojo	Prueba con cinta descrita en el manual de campo, §7.1. Un eco devuelto por las puntas del rastrojo da una altura plausible, estable y en rango, que el sistema marcaría como `buena`. **Hasta que la prueba la valide, `header_height` no se publica**
Que el GPS mida bien la velocidad, sobre todo a paso lento	Tramo medido con cinta, a ritmo constante, comparando con la media del GPS; y la máquina detenida, que debe dar 0 (manual de campo, §7.2)

Y nueve medidas de ingeniería que el documento da por estimadas:

Medida	Cómo se comprueba
Consumo real de cada tipo de nodo	Multímetro en serie, durante una hora de operación real
Autonomía real del paquete	Una jornada completa registrando la tensión del latido
Temperatura de la Pi bajo carga	Ocho horas al 70 % de CPU en la caja definitiva
Potencia y ventana del LED de la tolva	Banco, con la cámara, la distancia real y la exposición manual: la foto debe salir bien expuesta y **todas** las guardadas, iluminadas
Consumo real de la Pi y autonomía de la batería de cabina	Una jornada completa en fase de dataset, midiendo la tensión de la batería
Capacidad real de las pilas AA	Descarga completa de un juego: la caja dice 3600 mAh y el cálculo usa 2600
Luz y autonomía del LED a 9 V	Banco: exposición de la foto con la batería llena y casi vacía
Zona ciega y tensión del RCWL-1655	Banco, con cinta: no hay hoja de datos oficial
Tiempo hasta la primera solución y pérdidas de señal del GPS	Una jornada completa, contando los tramos con `sin_fix` en `telemetria`

Hasta que esas medidas estén hechas, las 11 horas de las cámaras (§3.2) y las 12 de la central (§3.5) son cálculos, no hechos.

---

11 · Resumen de decisiones y su porqué

Decisión	Por qué
Un ESP32 por sensor y por cámara (S3 en cámaras, C6 en sensores)	Cables cortos, fallos aislados, firmware simple
Nada conectado a la máquina	No hay que intervenir el cableado de una cosechadora ajena, ni pedir permiso para hacerlo
Batería propia por nodo	Un fallo eléctrico de la máquina no apaga los siete

4 AA Ni-MH directo al pin de 5 V	Pilas comunes y carga sencilla, sin elevador ni protección; a cambio, ≈ 11 h en las cámaras
LiFePO4 de 12 V para la central	Una sola pieza de ≈ 150 Wh, tolerante al calor; desde 12 V la Pi y el router toman ≈ 1 A
LED de tolva pulsado, con batería propia de 9 V	No toca las pilas de `CAM04`, y encenderlo solo durante la foto cuesta diez veces menos energía
Ahorro de radio solo en sensores	Las cámaras pagan latencia; los sensores no, porque el instante viaja en la muestra
Velocidad por GPS USB, sin Hall de rueda	El Hall no sobrevivía al agua y al lodo de la rueda, y la rueda patina. El GPS mide la velocidad sobre el suelo sin constantes que medir y sin un nodo más
La posición del GPS no se guarda	Solo hace falta la velocidad; no queda registro de dónde está cada lote
Central en cabina, AP fuera	La Pi quiere estar fresca, la antena cerca de los nodos
Router dedicado (Tenda N301) y no celular	Un hotspot de iPhone no admite ocho estaciones
SSD NVMe y no microSD	15–17 GB por jornada, escritos a diario
Sin microSD en los nodos	Todo va al SSD de la Pi; a cambio, si la Pi o el Wi-Fi caen, los frames de ese intervalo se pierden
Sin acelerador de IA	Decisión de proyecto; la CPU basta con ritmos escalonados
Segmentación semántica	Es lo que corresponde a un porcentaje de píxeles
NCNN	Cinco veces más rápido que PyTorch en esta máquina
Fase de dataset por defecto	Un sistema que arranca inventando números es peor que uno que calla
Valor nulo cuando no hay medida	Un cero inventado contamina el entrenamiento sin dejar rastro
`201` antes de inferir	La captura tiene prioridad sobre el diagnóstico
Hash recalculado en la central	Prueba el viaje, no solo lo que el nodo cree haber enviado
Degradación térmica deliberada	Elegir qué se pierde en vez de que lo elija el hardware